

中山大学

《软件工程》期末大作业

软件配置与运维文档

项目名称：逸仙活动云——中山大学校园活动云平台

小组成员：钟梓俊 邱剑波 张宸睿

指导老师：王可泽

2026 年 7 月 16 日

目录

1	文档概述	1
2	配置管理	1
2.1	配置项说明	1
2.2	配置文件管理	1
2.3	环境变量管理	2
2.4	敏感信息管理	2
3	版本管理	2
3.1	版本号规范	2
3.2	发布版本管理	3
3.3	变更记录	3
3.4	回滚机制	3
4	持续集成	3
4.1	CI 流程	3
4.2	自动化构建	4
4.3	自动化测试	4
4.4	构建产物管理	4
5	软件部署	5
5.1	环境要求	5
5.2	安装部署步骤	5
5.3	初始化配置	5
5.4	升级步骤	5
5.5	卸载步骤	6
6	运行维护	6
6.1	日志管理	6
6.2	数据备份	6
6.3	数据恢复	6
6.4	异常处理	7
6.5	故障排查	7
7	发布与回滚	7
8	运维风险与应急方案	8

1 文档概述

本文档定义逸仙活动云项目的配置管理、版本管理、持续集成、软件部署、运行维护及应急方案。目标是确保每个交付版本可识别、可重建、可部署、可监控和可恢复。文档适用于项目团队的开发和运维人员，以及后续接手项目维护的第三方开发者。文档中“已实现”指代码库已有对应实现；“计划”指建议在交付或运维阶段落实，二者不作混同。

2 配置管理

2.1 配置项说明

项目涉及的配置项按类别划分如下：

1. **应用配置**：前端 API 地址(VITE_API_BASE_URL)、后端数据库连接串(DATABASE_URL)、Redis 连接串 (REDIS_URL)、CORS 域名。
2. **安全配置**：JWT 签名密钥 (JWT_SECRET_KEY)、密码哈希算法、验证码有效期。
3. **外部服务配置**：邮件 SMTP 服务器地址与凭据、外部搜索 API 密钥、AI 服务 API 端点和密钥、数据源抓取间隔与超时。
4. **部署配置**：容器镜像版本、数据卷挂载路径、Gunicorn Worker 数量、健康检查端点。

2.2 配置文件管理

项目通过分层配置文件管理各类配置：

1. **前端**：`.env.example` 作为模板文件提交到仓库，开发者复制为 `.env.local` 后填充实际值。Vite 在开发阶段自动加载根目录的 `.env.local` 和环境变量。
2. **后端**：`.env.example` 作为模板提交到仓库，开发者复制为 `.env` 后填充。Flask 通过 `python-dotenv` 在应用启动时加载。
3. **Docker Compose**：`docker-compose.yml` 定义服务拓扑和卷挂载,通过 `environment` 字段或 `env_file` 引用环境变量文件。

所有配置文件模板不包含真实密钥和密码。开发环境、测试环境和生产环境使用各自独立的环境文件，互不干扰。

2.3 环境变量管理

环境	配置来源	数据与依赖
本地开发	项目根目录 <code>.env.local</code> （前端）或 <code>.env</code> （后端）	Mock 服务或 SQLite 数据库；不连接生产 Redis 或邮件服务。
测试/CI	GitHub Actions Secret 或临时环境变量	内存 SQLite；禁用真实邮件、Redis 和外部 LLM。
预发布	独立 <code>.env</code> 文件或容器编排工具注入	独立 PostgreSQL 和 Redis 实例；脱敏样本数据。
生产	受控 Secret 管理服务（如 Docker Secret 或云平台密钥管理）	PostgreSQL 主从、Redis 集群；HTTPS、备份和监控必须启用。

2.4 敏感信息管理

- 密钥分类：** JWT 签名密钥、邮件 SMTP 密码、AI/搜索 API 密钥、数据库连接密码。
- 存储原则：** 敏感信息以环境变量或 Secret 管理服务注入容器，不写入代码仓库、日志文件或截图。
- 泄露处置：** 发现泄露后立即撤销原密钥、生成新密钥、轮换受影响范围内的所有凭据、审计泄露时间窗口内的访问记录、重建受影响的制品。
- 开发默认值：** `.env.example` 中的默认凭据仅用于本地演示环境，生产部署前必须替换为高熵随机值。

3 版本管理

3.1 版本号规范

项目使用语义化版本号 `vMAJOR.MINOR.PATCH`：

- MAJOR：** 破坏性接口变更或数据库 Schema 不向后兼容时递增。
- MINOR：** 新增向后兼容的功能时递增。
- PATCH：** 缺陷修复或内部优化时递增。

前端和后端仓库独立标识版本。发布时以兼容版本对组成系统基线，同时记录前端 tag、后端 tag、接口契约版本和数据库 Schema 版本。

3.2 发布版本管理

1. 主分支（main）始终保持可构建状态，所有变更通过合并请求合入。
2. 功能分支在短生命周期内开发，完成后删除。
3. 每个可交付版本从 main 创建 tag，tag 名使用 `vMAJOR.MINOR.PATCH`。
4. 紧急修复从受影响 tag 创建 hotfix 分支，修复后同时合回 main 和发布分支。

3.3 变更记录

所有版本变更记录在 release notes 中，包含以下信息：

1. 版本号和发布日期。
2. 新增功能清单及关联需求编号（FR-xx）。
3. 修复的缺陷清单及关联缺陷编号。
4. 接口或数据库 Schema 变更摘要。
5. 升级注意事项（如有不兼容变更）。

3.4 回滚机制

1. **Docker 部署回滚：**停止当前容器，使用上一版本的镜像重新启动，恢复上一版本的数据库备份。
2. **数据库回滚：**在迁移前执行数据库全量备份；回滚时恢复备份并验证数据完整性。
3. **前端回滚：**将反向代理或 Web 服务器的静态资源目录切换回上一版本的构建产物。

回滚后保留当前版本和回滚版本的日志、镜像摘要和配置快照，用于后续根因分析。

4 持续集成

4.1 CI 流程

前端仓库已配置 GitHub Actions CI 工作流（`.github/workflows/ci.yml`），在 push 和 pull request 事件上触发。工作流执行以下步骤：

1. 检出代码并配置 Node 22 环境（`actions/setup-node@v4`）。
2. 执行 `npm ci` 安装精确版本依赖。
3. 安装 Chromium 浏览器（`npx playwright install --with-deps chromium`）。
4. 执行 `npm run build`（类型检查 + 生产构建）。

5. 执行 `npm run test:unit` (Vitest 单元测试)。
6. 执行 `npm run test:e2e` (Playwright 端到端测试)。

4.2 自动化构建

前端通过 `npm run build` 触发 `vue-tsc -b && vite build`, 串行执 TypeScript 类型检查和 Vite 生产构建, 输出可部署的静态资源到 `dist/` 目录。后端通过 Dockerfile 定义构建过程, 安装 Python 依赖并打包 Flask 应用。构建失败时 CI 工作流终止并报告错误。

4.3 自动化测试

1. **单元测试:** Vitest (前端) 覆盖组件渲染、状态管理和工具函数; pytest (后端) 覆盖 API、服务和业务规则。
2. **端到端测试:** Playwright 覆盖访客浏览、用户注册登录、活动搜索等关键用户旅程。
3. **聚合检查:** `npm run verify` 串行执行构建、单元测试和端到端测试, 作为提交前的完整性验证。

4.4 构建产物管理

前端构建产物为静态文件 (HTML/CSS/JS), 直接部署到 Web 服务器或反向代理。后端构建产物为 Docker 镜像, 推送到镜像仓库后由 Compose 引用。发布制品包括前端静态构建包、后端不可变镜像、SBOM 依赖清单、测试报告和 release notes, 以 tag 和 commit SHA 双重标识。

5 软件部署

5.1 环境要求

项目	要求
操作系统	Linux（推荐 Ubuntu 22.04+） / Windows / macOS（仅开发）
运行环境	Docker Engine 24+、Docker Compose v2
数据库	PostgreSQL 15+（部署） / SQLite 3（开发）
缓存	Redis 7+（可选，部署推荐启用）
反向代理	Nginx（生产，建议配置 TLS）
网络	容器间内网通信；生产环境需开放 443（HTTPS）

5.2 安装部署步骤

以下步骤涉及前端和后端两个仓库，请根据提示在对应目录下执行：

1. 后端目录：复制环境变量模板并填写配置：`cp .env.example .env`。
2. 后端目录：执行 `docker compose up -d --build` 启动所有服务（API、PostgreSQL、Redis、Celery Worker/Beat）。
3. 访问 GET `http://127.0.0.1:5000/api/health` 确认 API 存活。
4. 前端目录：执行 `npm run build` 构建前端静态资源。
5. 将前端构建产物 `dist/` 目录部署到 Nginx 或反向代理的静态资源路径。
6. 验证登录、活动查询和审核等最小业务流程正常。

5.3 初始化配置

首次启动时，Flask 应用通过 `app.config[SQLALCHEMY_TRACK_MODIFICATIONS]` 和模型注册自动创建数据库表。此方式仅适用于开发和演示环境，生产环境必须使用受控迁移工具（如 Alembic）管理 Schema 变更。系统同时提供开发用初始数据（默认管理员账号和示例活动），首次部署后应修改默认密码。

5.4 升级步骤

1. 拉取目标版本的 Docker 镜像或前端构建产物。
2. 备份当前数据库（`pg_dump`）和配置文件。

3. 执行数据库迁移脚本（如有 Schema 变更）。
4. 更新 Docker 镜像版本标签并重启服务：`docker compose up -d --pull always`。
5. 执行冒烟测试验证升级后业务正常。

5.5 卸载步骤

1. 停止所有容器：`docker compose down`。
2. 删除数据卷（谨慎操作，将丢失所有数据）：`docker volume rm <volume_name>`。
3. 删除镜像：`docker rmi <image_name>:<tag>`。
4. 删除项目目录和配置文件。

6 运行维护

6.1 日志管理

1. **日志来源：**Flask 应用日志（请求、错误、审计）、Celery Worker 日志（任务执行）、Nginx 访问日志、PostgreSQL 慢查询日志。
2. **日志内容规范：**日志不得包含密码、JWT、完整邮箱验证码或完整搜索词。审核和管理操作必须记录操作者、动作、目标和时间。
3. **日志轮转：**使用 Docker 的日志驱动或 logrotate 工具实现日志轮转，避免磁盘空间耗尽。

6.2 数据备份

1. **备份策略：**PostgreSQL 数据库执行每日全量备份，结合 WAL 日志实现增量恢复。
2. **备份存储：**备份文件加密后存储于异地或独立存储系统，与生产环境隔离。
3. **验证：**定期执行恢复演练，确保备份文件可正常恢复且数据完整性通过校验。

6.3 数据恢复

1. 确认故障范围和影响程度，隔离故障环境。
2. 选择最近的可用备份文件，执行恢复操作。
3. 恢复后验证核心数据完整性（用户数、活动数、关系数据）。
4. 确认服务健康检查通过后，重新开放服务。

6.4 异常处理

异常类型	处理措施	告警方式
API 服务不可用	检查容器状态、应用日志和数据库连接；重启服务或回滚到上一版本。	健康检查连续失败触发告警。
数据库连接失败	检查 PostgreSQL 容器、数据卷和网络；从备份恢复数据卷。	应用日志记录连接错误并告警。
Redis 不可达	确认 Redis 容器运行状态；限流和队列功能降级，核心 API 继续运行。	日志记录降级状态，待连接恢复后自动重连。
任务队列积压	检查 Worker 进程和外部依赖；暂停失败率高的数据源。	监控队列深度，超过阈值触发告警。
安全事件	撤销泄露的凭据，保全日志，评估数据影响范围，修复漏洞。	异常登录或越权行为记录审计日志。

6.5 故障排查

1. 容器级别：使用 `docker ps` 检查容器运行状态，`docker logs <container>` 查看应用日志。
2. 应用级别：访问 `/api/health` 检查 API 存活状态；查看 Flask 日志定位 5xx 错误；检查数据库连接池和慢查询。
3. 网络级别：使用 `docker network inspect` 检查容器间通信；确认反向代理配置和防火墙规则。
4. 数据级别：检查数据库表结构完整性；确认外键约束和唯一约束未损坏；检查数据卷挂载状态。

7 发布与回滚

发布流程遵循以下步骤：

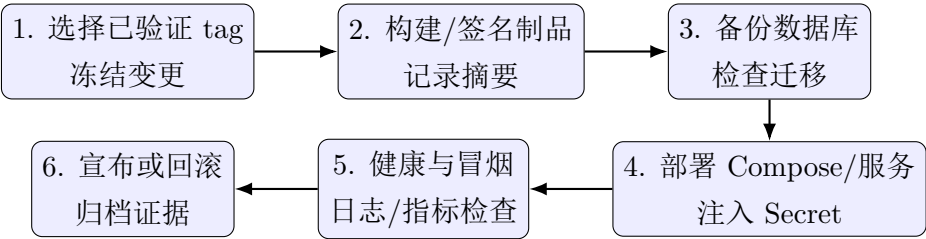


图 1：发布与回滚流程

发布前检查清单：

- 1. 选择已通过质量门禁的前后端 commit/tag，冻结接口和数据库迁移版本。
- 2. 确认无真实密钥、.env 文件或本地数据进入制品。
- 3. 在预发布环境恢复备份、执行迁移和冒烟测试。
- 4. 复核 TLS 配置、CORS 设置、请求体限制、健康检查和日志脱敏。
- 5. 部署后验证健康接口、登录、活动浏览、发布审核和审计查询。

回滚触发条件：健康检查连续失败、核心冒烟测试未通过、错误指标超过阈值。回滚时停止当前服务，恢复上一版本镜像和数据库备份，保留日志和版本信息用于根因分析。

8 运维风险与应急方案

风险	影响	应急方案
数据库损坏或数据丢失	用户数据不可恢复，核心业务中断	从每日备份恢复；启用只读维护页面；排查损坏原因。
密钥泄露	未授权访问、数据泄露	立即撤销并轮换密钥；审计泄露时间窗口内操作；通知受影响用户。
外部服务不可用	邮件发送、搜索、AI 功能降级	切换至降级模式；保留错误日志待服务恢复后重试。
容器运行时故障	服务中断	检查 Docker 守护进程状态；重新启动容器；切换至备用节点。

当前代码提供 Docker Compose 和健康检查入口，但尚未配置生产环境的 TLS、集中监控、自动备份、分支保护或灾备演练。以上运维措施列为上线前必做项，在课程项目阶段以可复现的部署拓扑和操作文档作为交付基础。每次运行事故由值班人员先止损和保全日志，再由开发、测试、运维人员共同完成根因分析、修复、回归验证和预防行动跟踪。